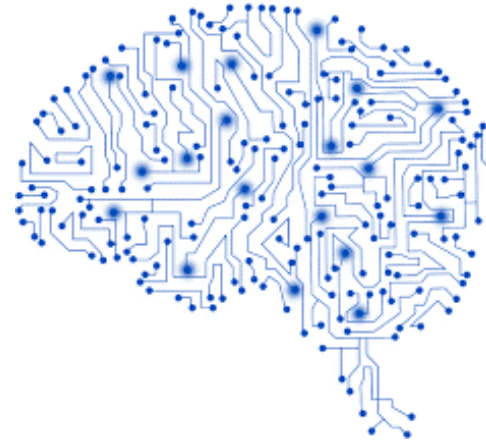




University of Minho
School of Engineering



Dados e Aprendizagem Automática

Support Vector Machines and Librosa

DAA 2025/2026

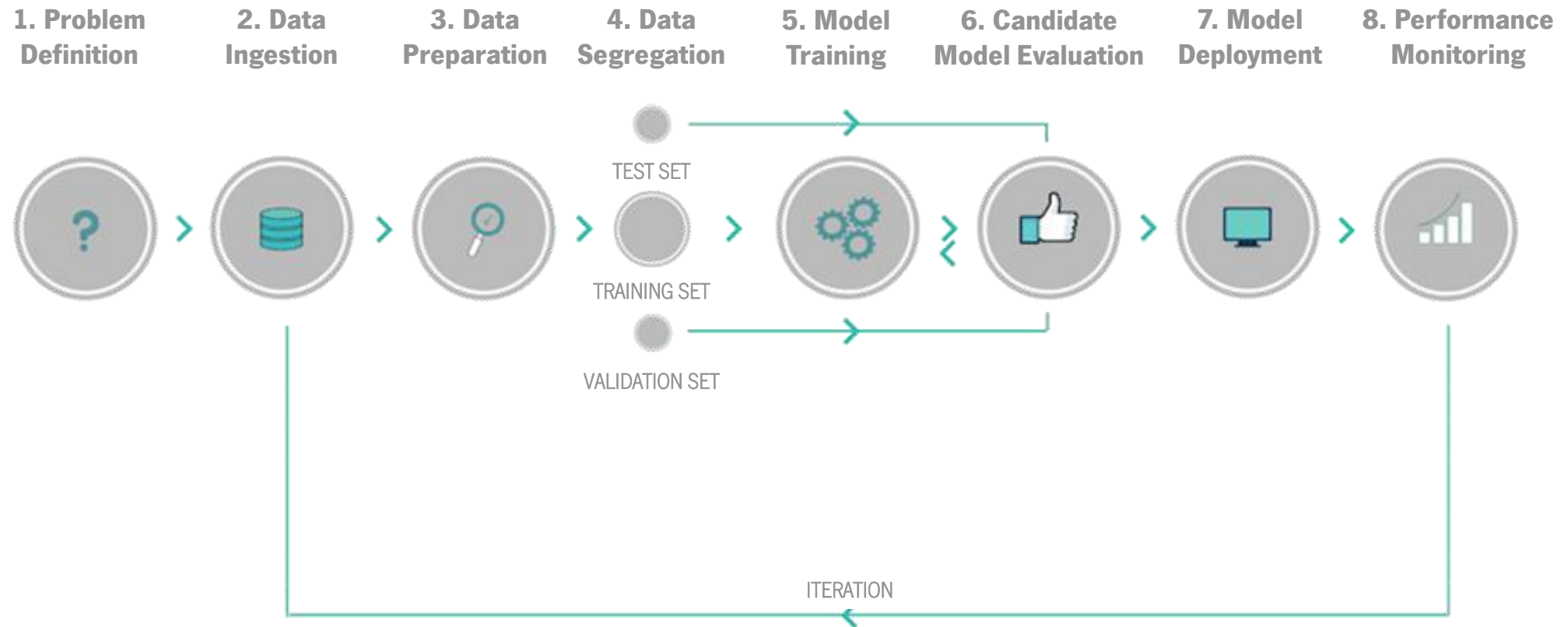
Filipa Ferraz, Victor Alves, Dalila Durães, Francisco Marcondes, Guilherme Barbosa

Part X

Contents

2

- Librosa
- Support Vector Machines
- Hands On





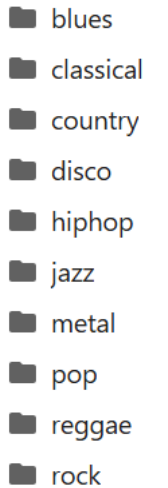
Librosa

The Data

5

Dataset: [GenAudio](#) comprises audio files and CSV files containing extracted features from those audio files.

- **genres_original** – a collection of 10 genres, each containing 100 files of 30 seconds;
- **CSV files** (*features_3_sec.csv* and *features_30_sec.csv*) – files that contain extracted features from the audio files. One CSV includes the mean and variance computed over multiple features that can be obtained from each 30-second song. Another CSV has the same structure, although the songs were divided into 3-second segments, which substantially increases the amount of data used in the classification models.



These files are part of the [GTZAN](#) dataset, which is widely used for evaluating Music Genre Recognition (MGR) in machine learning research. The data were collected for a university team project between 2000 and 2001 from various sources, including personal CDs, radio recordings and microphone recordings, in order to ensure diversity in the types of audio captured.

Librosa

6

Librosa is a python library used for audio and music analysis. It is a powerful tool that is widely used for audio visualisation and for developing Music Information Retrieval (MIR) systems.



You may need to install [librosa](#):
`pip install librosa`

Explore the sound files

7

Start by selecting a file and exploring it.

```
audio_path = 'Data/genres_original/reggae/reggae.00010.wav'
```

Load the audio as a waveform (data) and store the sampling rate (sr).

```
data, sr = librosa.load(audio_path)
print(type(data), type(sr))

print(data.shape, sr)
```

```
<class 'numpy.ndarray'> <class 'int'>
(661794,) 22050
```

Set the sample rate to 45600 to obtain the signal value array.

```
librosa.load(audio_path, sr=45600)
```

```
(array([-0.00555292, -0.00768963, -0.00668519, ...,  0.08035275,
        0.0663713 ,  0.03239053], dtype=float32),
45600)
```

Understanding waveforms, sampling rates and other sound concepts: [here](#).

Explore the sound files

8

Convert the signal using Short-Time Fourier Transform (STFT).

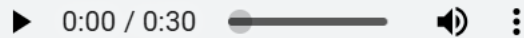
```
stft = librosa.stft(data)
```

Convert the amplitude spectrogram to a dB-scaled spectrogram.

```
stft_db = librosa.amplitude_to_db(abs(stft))
```

Then, play the audio file.

```
IPython.display.Audio(data, rate=sr)
```

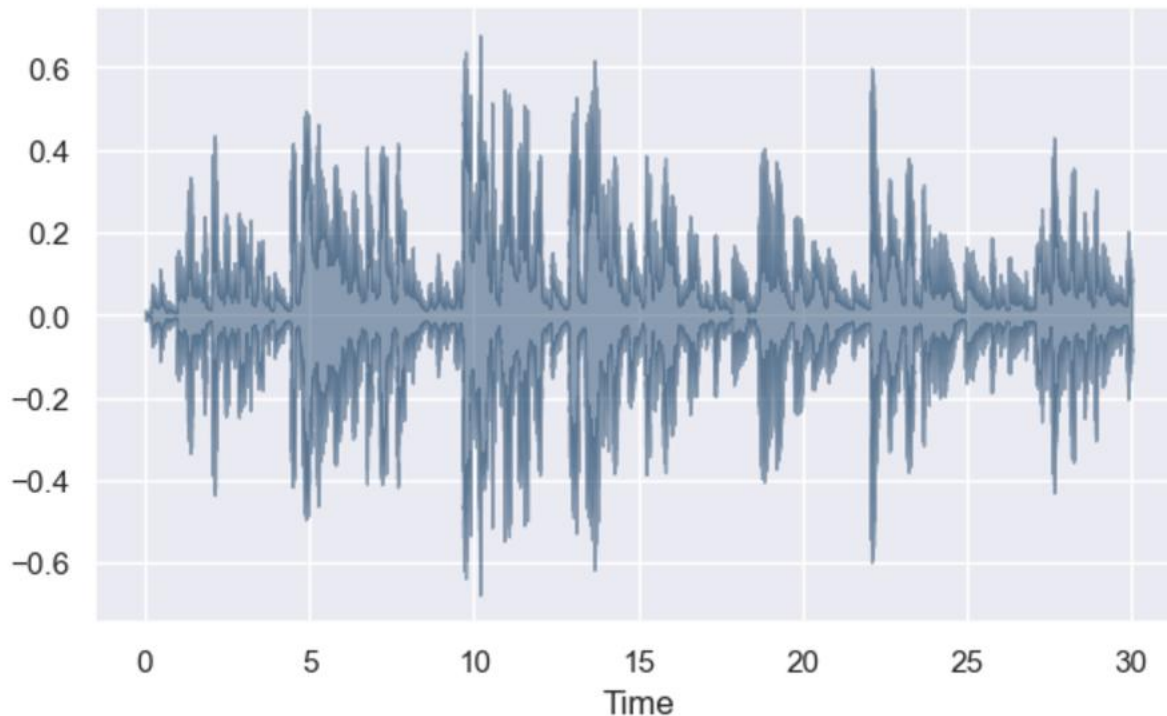


Explore the sound files

9

Take a look at the waveform representation of this audio file.

```
plt.figure(figsize=(7, 4))  
librosa.display.waveshow(data, color="#2B4F72", alpha=0.5)  
plt.show()
```



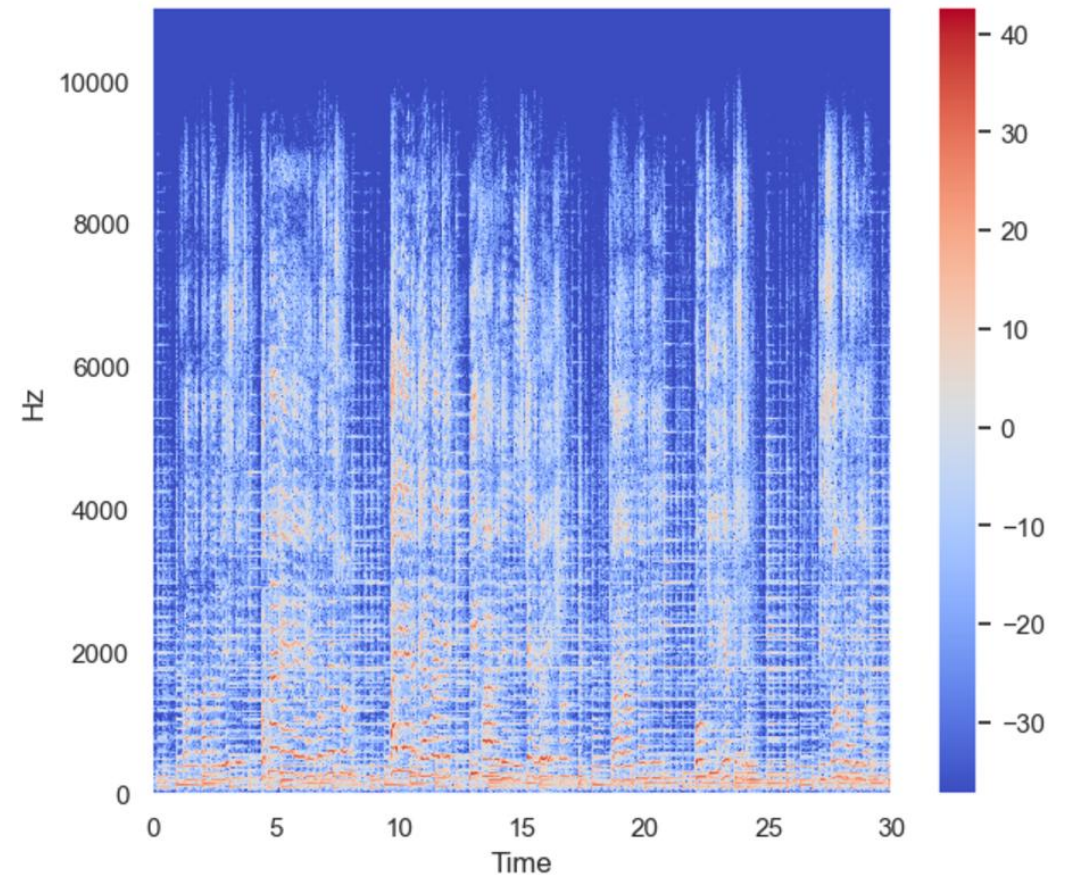
Explore the sound files

10

A *spectrogram* is a visual representation of signal loudness at different frequencies over time. It can be used to examine increases or decreases in energy over a period of time. Spectrograms are also known as sonographs, voiceprints and voicegrams.

Obtain a spectrogram of this audio file.

```
plt.figure(figsize=(7, 6))
librosa.display.specshow(stft_db, sr=sr, x_axis='time', y_axis='hz')
plt.colorbar()
```



Data Pre-Processing

11

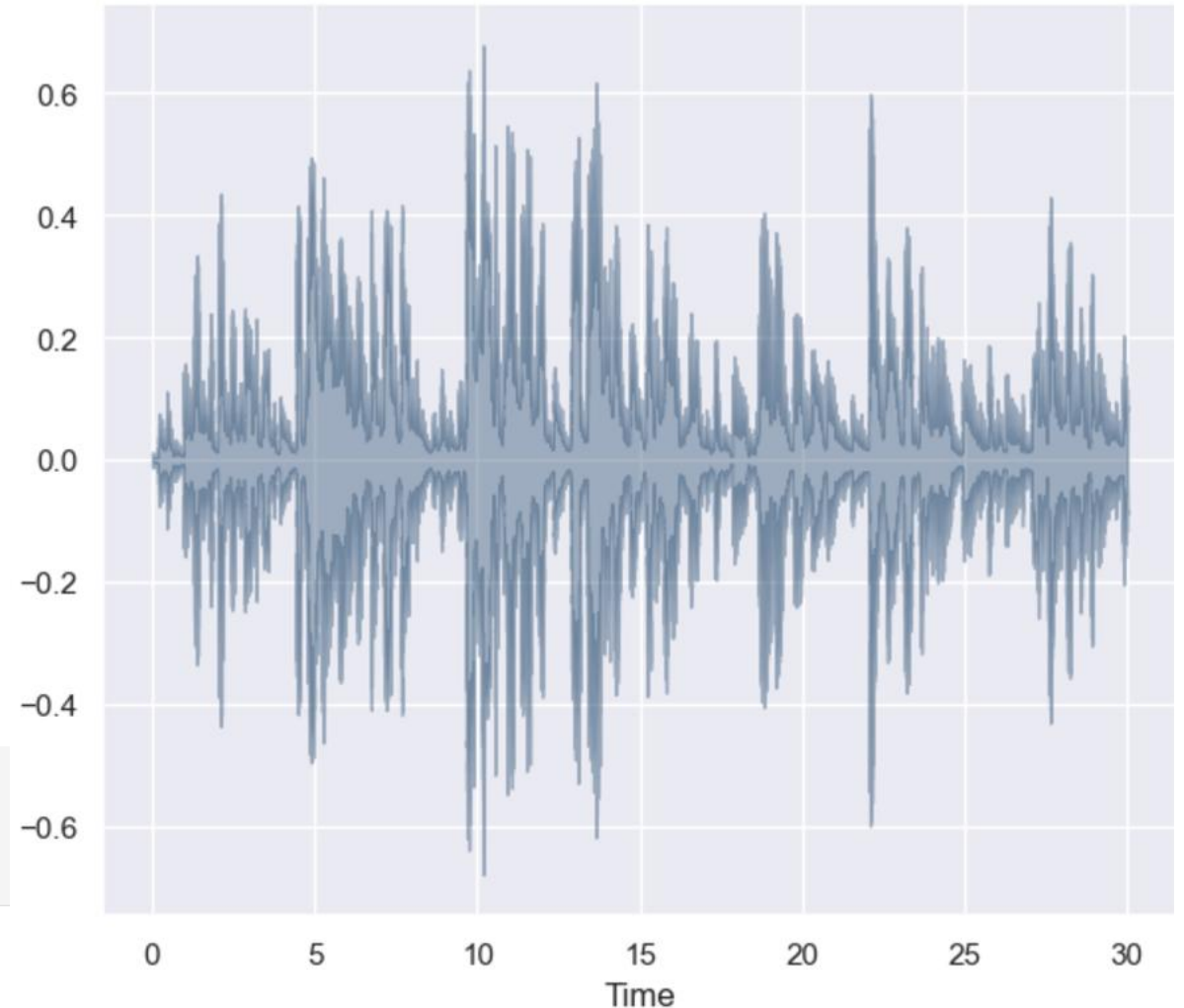
Extracting audio features

Each audio signal consists of various features. However, it must be extracted those that are relevant to the problem at hands. We will analyse some of these features.

- **Spectral Roll-Off**

It computes the roll-off frequency for each frame in each signal. This is the frequency at which a certain percentage (the *cut-off* point) of the total energy of a spectrum is obtained. It can be used to differentiate between the harmonic and noisy sounds.

```
spectral_rolloff = librosa.feature.spectral_rolloff(y=data, sr=sr)[0]  
  
plt.figure(figsize=(7, 6))  
librosa.display.waveshow(data, sr=sr, alpha=0.4, color="#2B4F72")
```



Data Pre-Processing

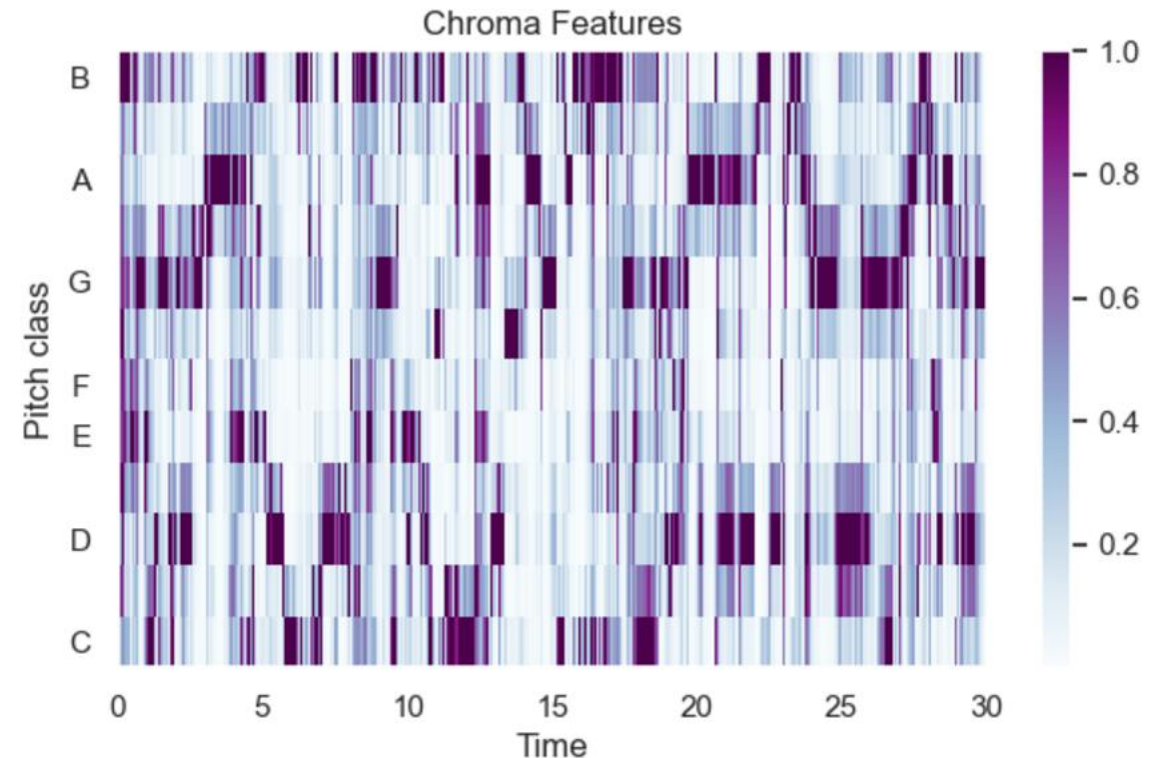
12

- **Chroma Feature**

It is closely related to the 12 different pitch classes. Chroma-based features are also known as pitch class profiles. They are a powerful tool for analysing and categorising music. Harmonic and melodic characteristics of music are captured by them. They compute the chromogram from a waveform or power spectrogram.

```
chroma = librosa.feature.chroma_stft(y=data, sr=sr)
```

```
plt.figure(figsize=(7, 4))  
lplt.specshow(chroma, sr=sr, x_axis="time", y_axis="chroma", cmap="BuPu")  
plt.colorbar()  
plt.title("Chroma Features")  
plt.show()
```



Data Pre-Processing

13

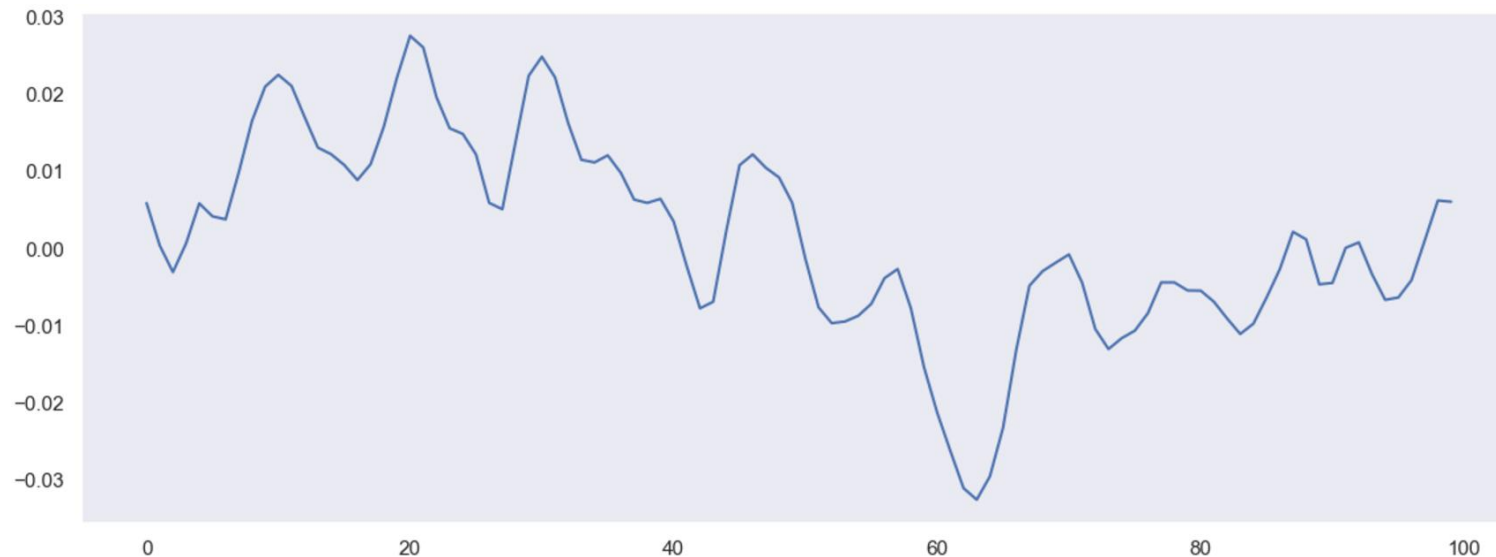
- **Zero-Crossing Rate**

The Zero-Crossing Rate (ZCR) is the rate of sign-changes along with the signal, i.e., the rate at which the signal changes from positive to negative or vice versa - the number of times the signal crosses x-axis. This feature is widely used in speech recognition and music information retrieval. It tends to be higher for highly percussive sounds, such as those in metal and rock music.

```
n0 = 9000
n1 = 9100
plt.figure(figsize=(14, 5))
plt.plot(data[n0:n1], color="#2B4F72")
plt.grid()

zcr = librosa.zero_crossings(data[n0:n1], pad=False)
print(sum(zcr))
```

10



Data Pre-Processing

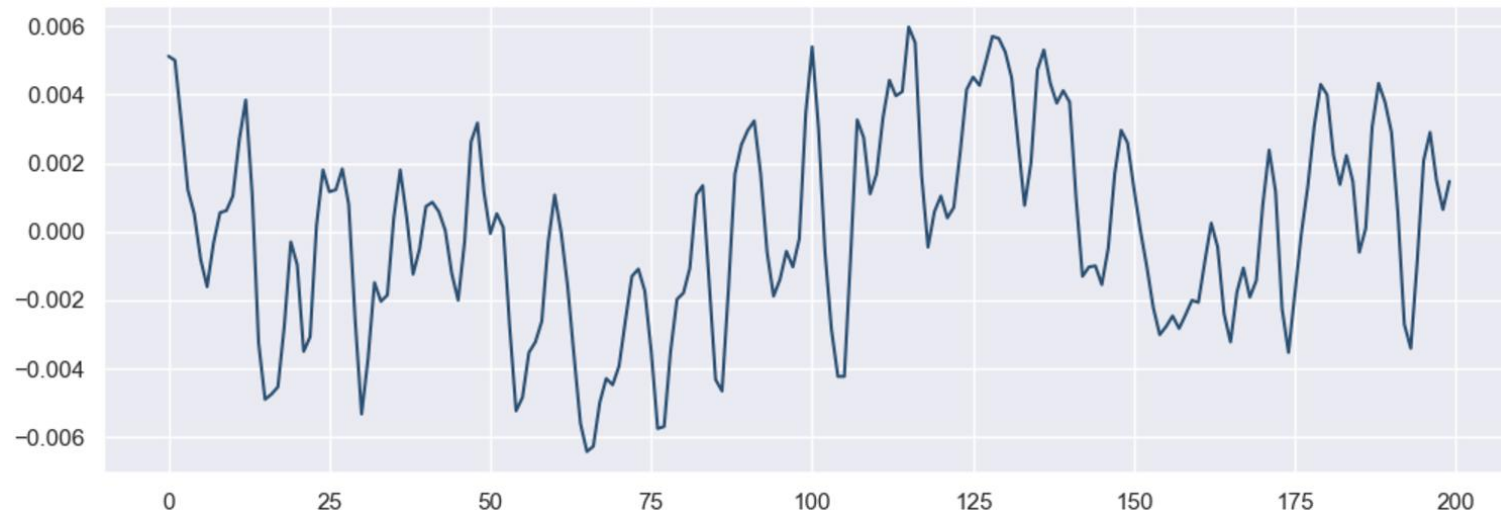
14

Change the start and end points.

```
start = 1000
end = 1200
plt.figure(figsize=(12, 4))
plt.plot(data[start:end], color="#2B4F72")

zcr = librosa.zero_crossings(data[start:end], pad=False)
print(sum(zcr))
```

36



Data Pre-Processing

15

- **Spectral Centroid**

It indicates where the "center of mass" of a sound is located and is calculated as the weighted mean of the frequencies present in the sound. Consider two songs: one from the blues genre and one from the metal genre. While the blues song has the same frequencies throughout, the metal song has more towards the end. Therefore, the spectral centroid of the blues song will lie near the middle of its spectrum, whereas that of the metal song will lie towards the end. To compute the spectral centroid, each frame of a magnitude spectrogram is normalised and treated as a distribution over frequency bins, from which the mean (centroid) is extracted per frame.

```
spectral_centroids = librosa.feature.spectral_centroid(y=data, sr=sr)[0]  
spectral_centroids.shape  
  
(1293,)
```

Convert the frame counts into seconds.

```
frames = range(len(spectral_centroids))  
t = librosa.frames_to_time(frames)
```

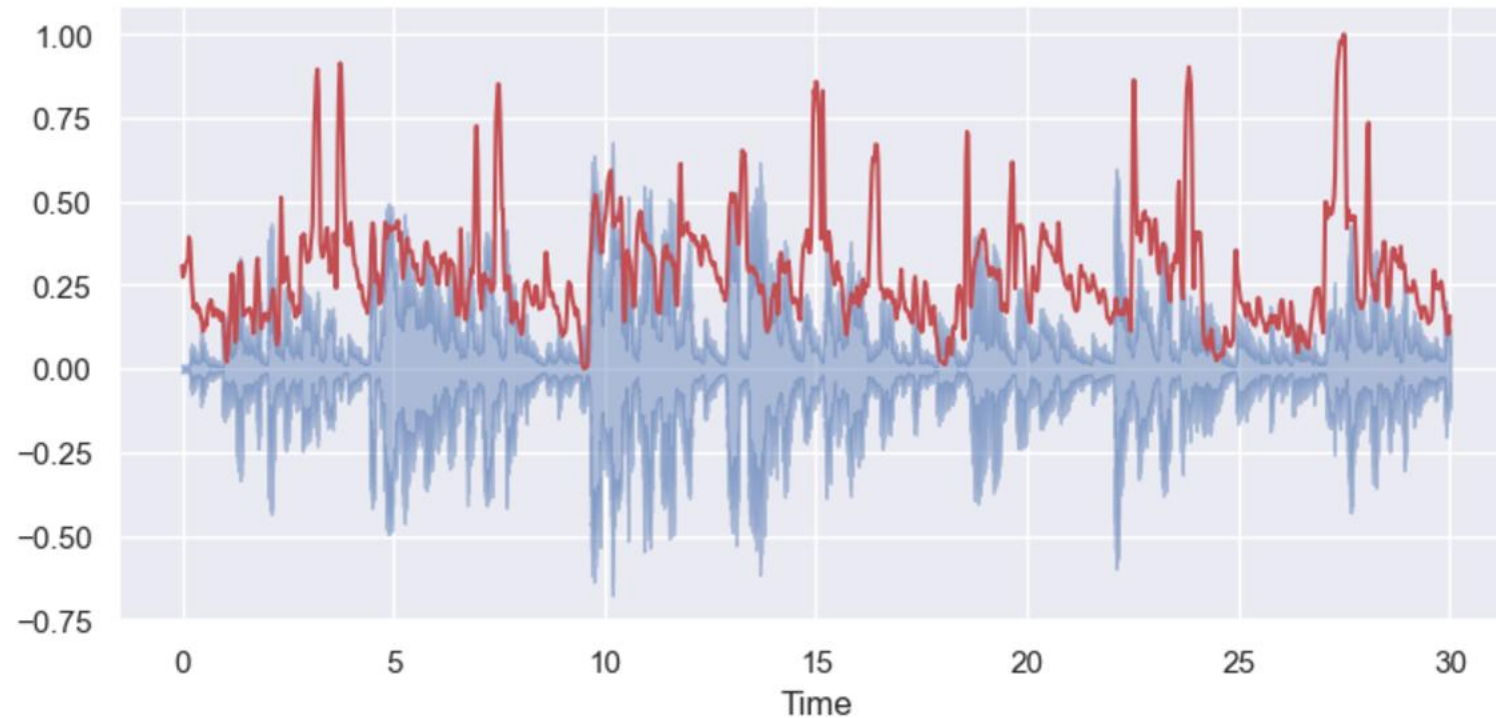

Data Pre-Processing

16

Transform the features by scaling each one to a given range.

```
def normalize(data, axis=0):  
    return sklearn.preprocessing.minmax_scale(data, axis=axis)
```

```
librosa.display.waveshow(data, sr=sr, alpha=0.4)  
plt.plot(t, normalize(spectral_centroids), color='r')
```



Exploratory Data Analysis

ipd.Audio(audio1)

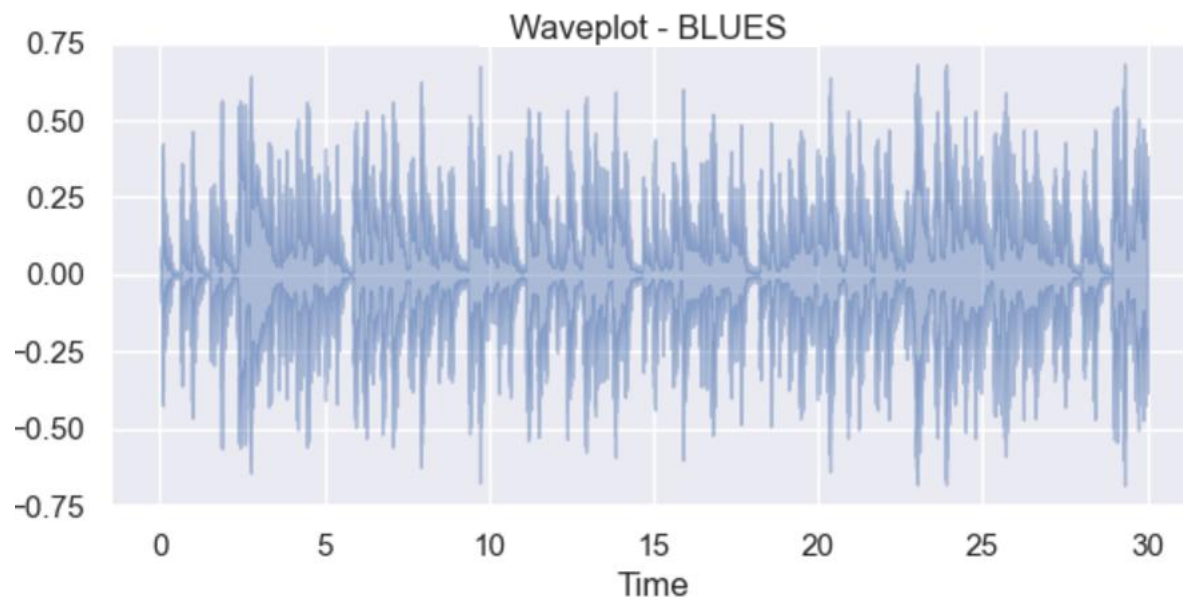
▶ 0:00 / 0:30 ———— 🔊 ⋮

17

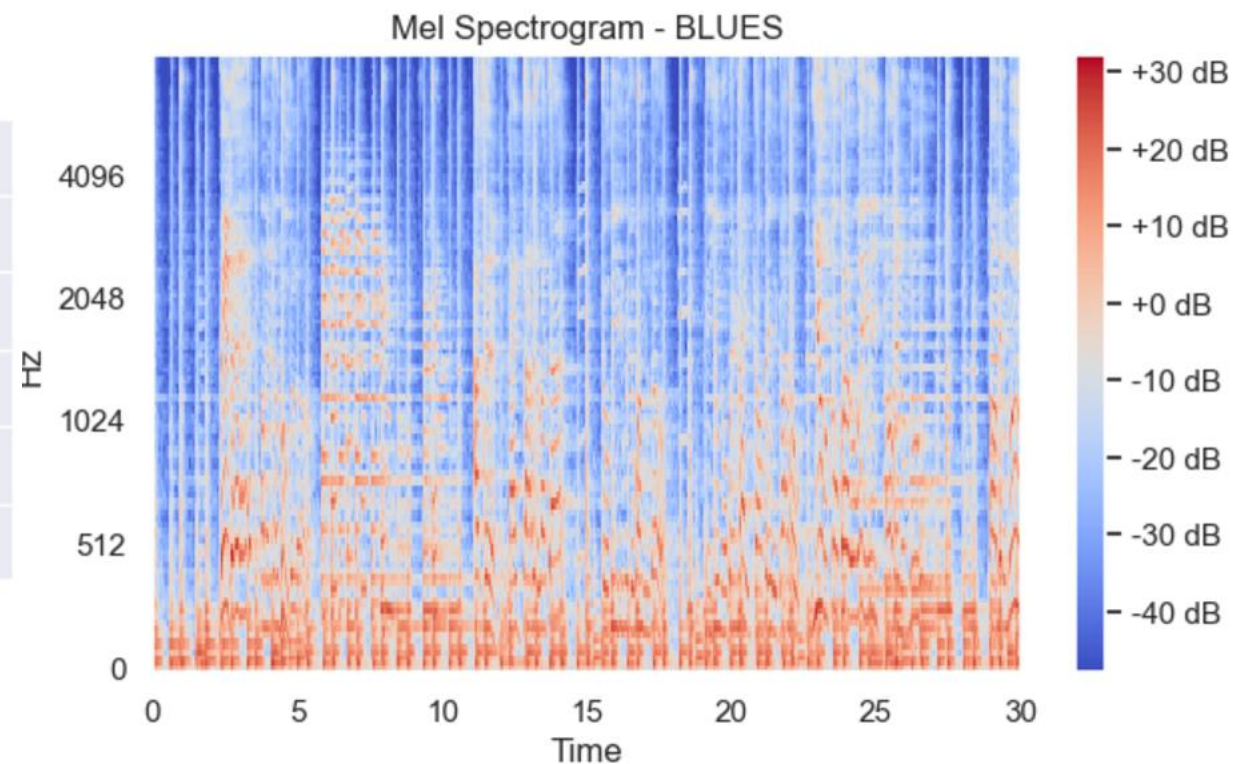
Visualise the wave plots and spectrograms of the audio files for all ten genre classes.

1. Blues

```
audio1 = 'Data/genres_original/blues/blues.00001.wav'  
data, sr = librosa.load(audio1)  
plt.figure(figsize=(7, 3))  
librosa.display.waveshow(data, sr=sr, alpha=0.4,)  
plt.title('Waveplot - BLUES')
```



```
plt.figure(figsize=(7, 4))  
spectrogram = librosa.feature.melspectrogram(y=data, sr=sr, n_mels=128, fmax=8000)  
spectrogram = librosa.power_to_db(spectrogram)  
librosa.display.specshow(spectrogram, y_axis='mel', fmax=8000, x_axis='time')  
plt.title('Mel Spectrogram - BLUES')  
plt.colorbar(format='%+2.0f dB')
```

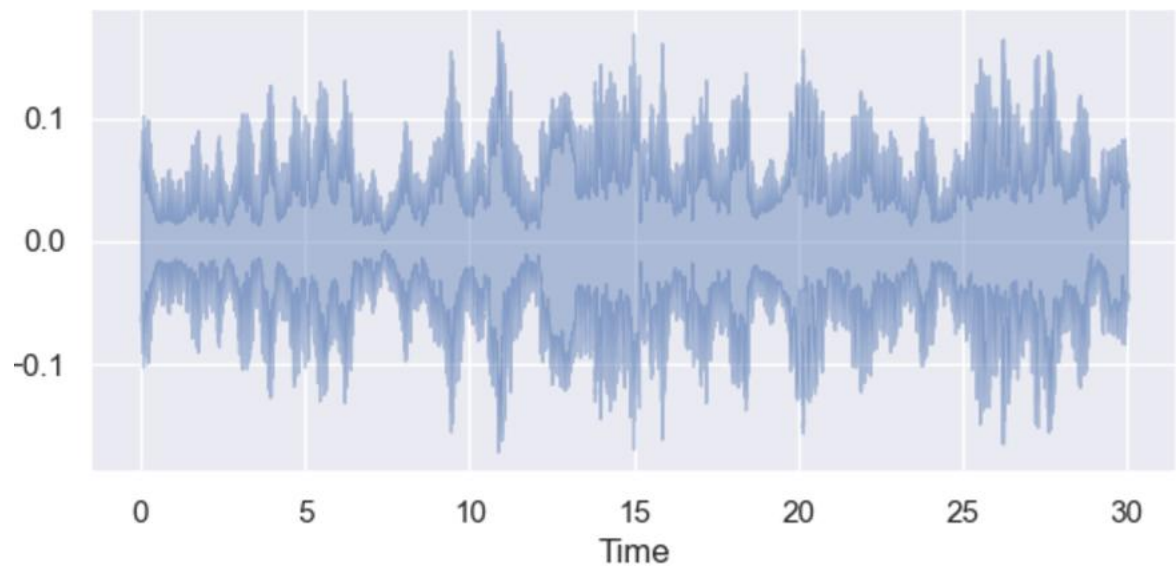


Exploratory Data Analysis

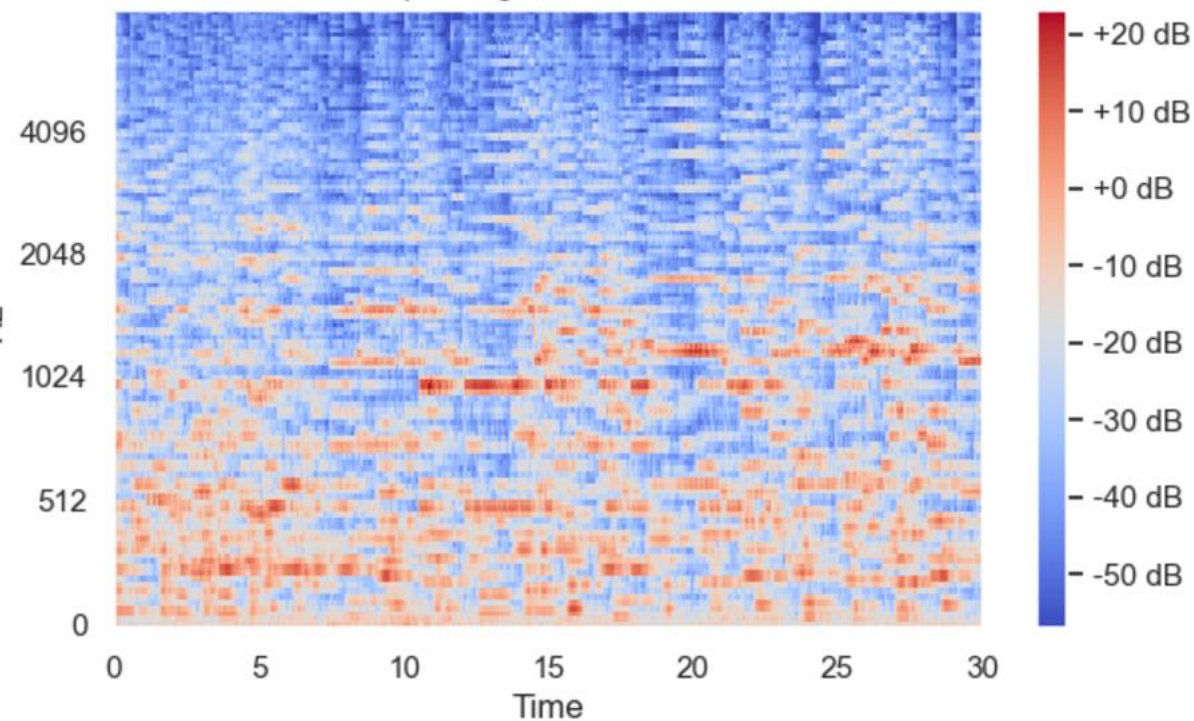
18

2. Classical

Waveplot - CLASSICAL



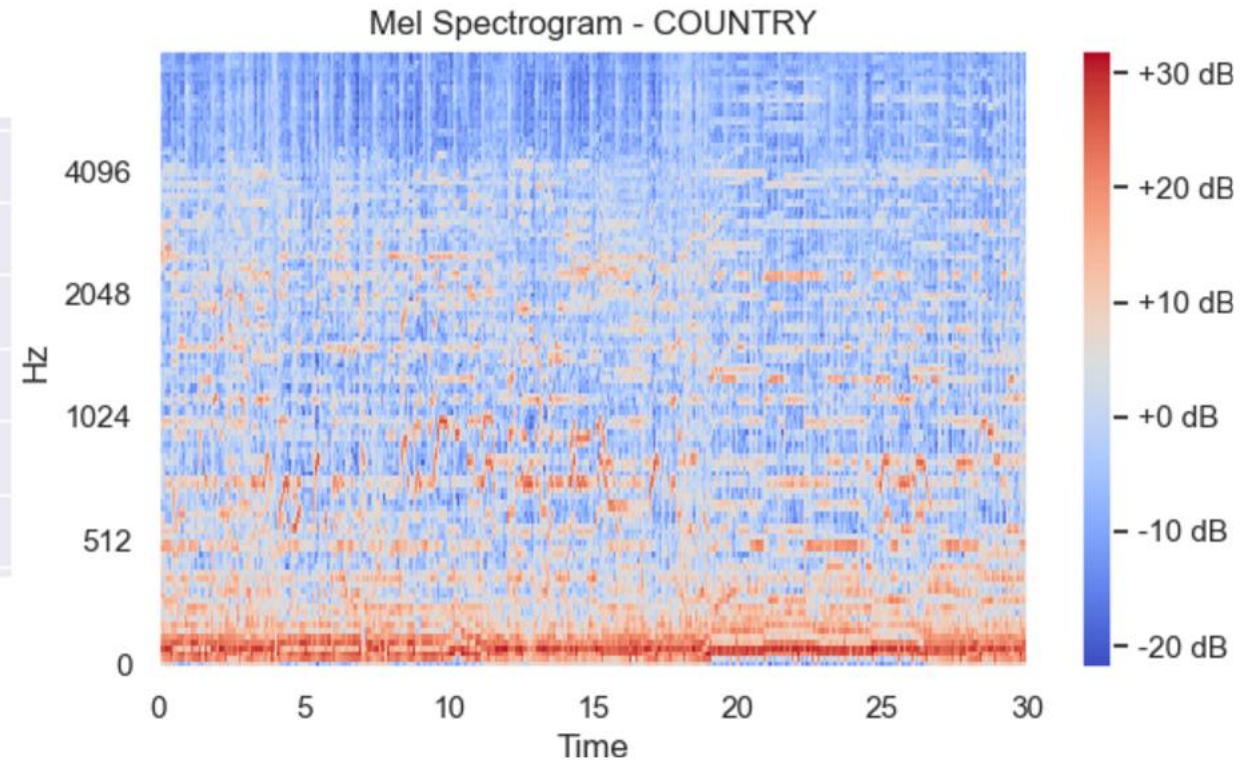
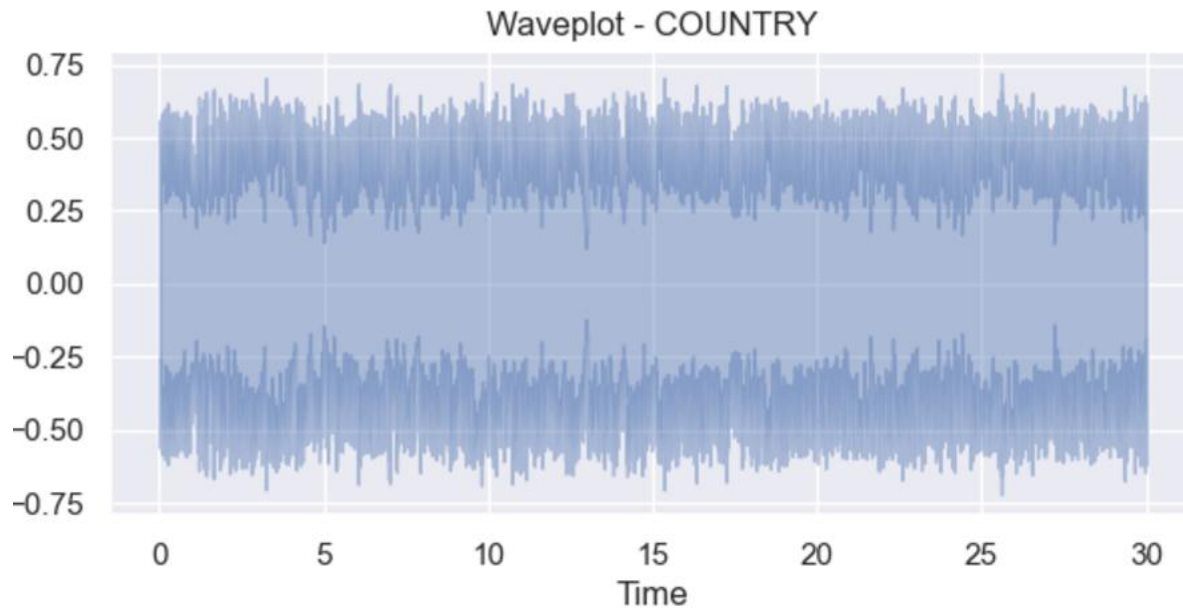
Mel Spectrogram -CLASSICAL



Exploratory Data Analysis

19

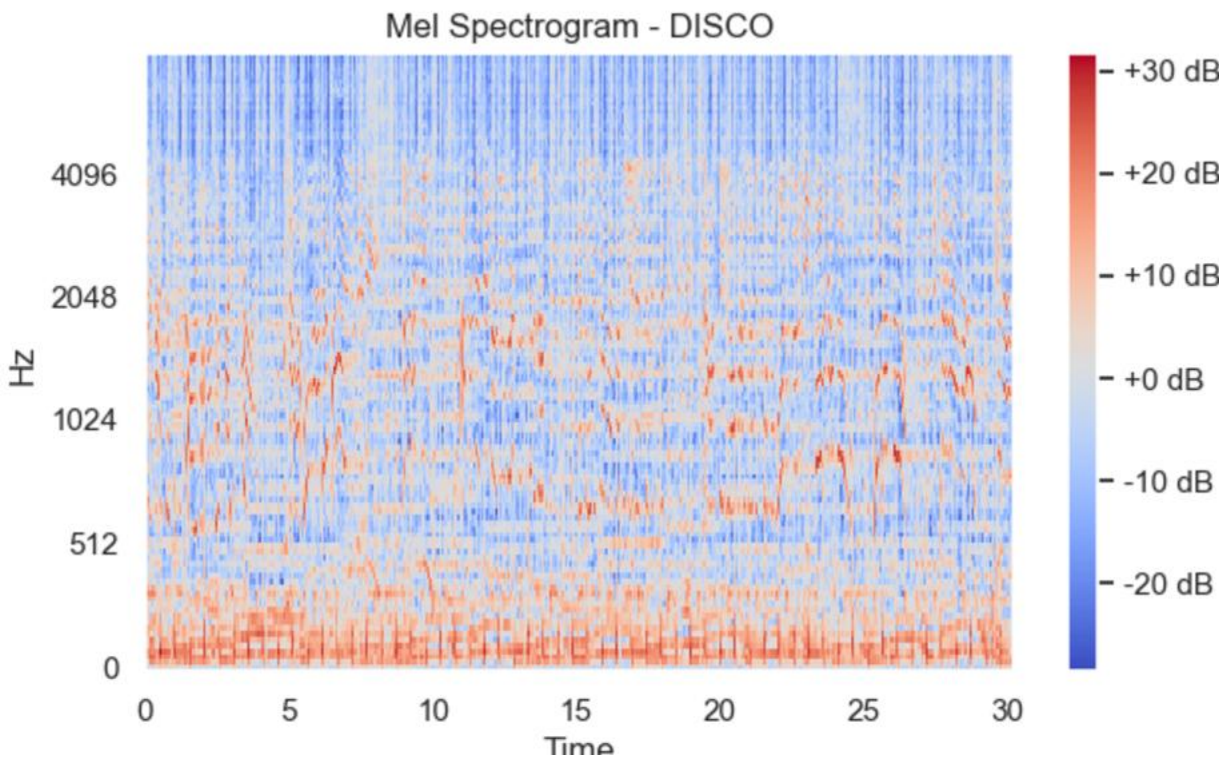
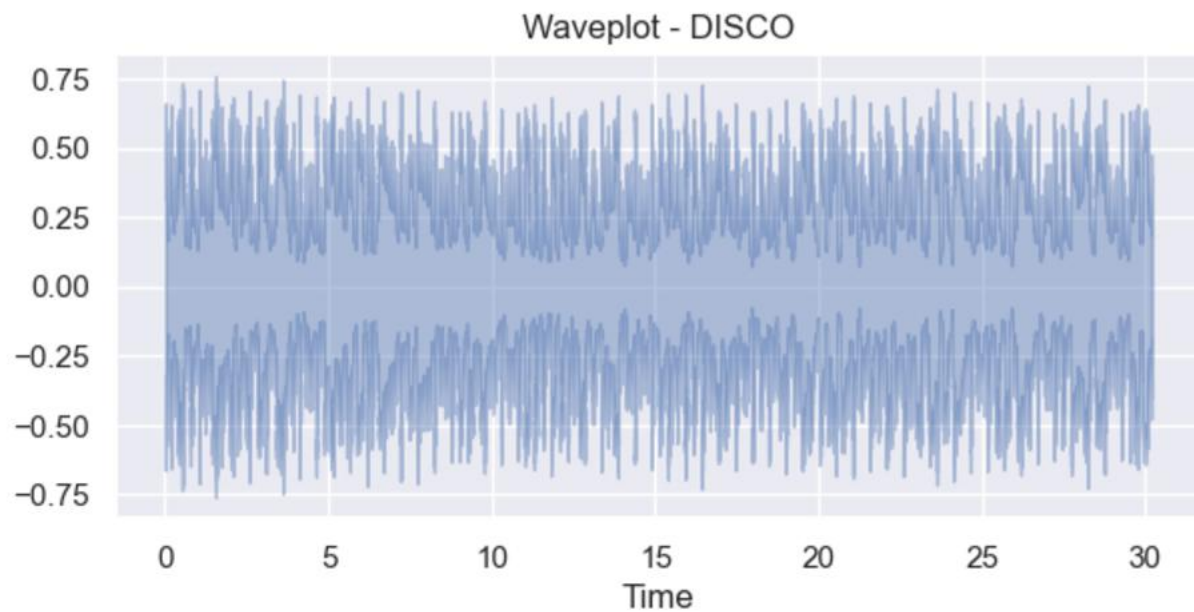
3. Country



Exploratory Data Analysis

20

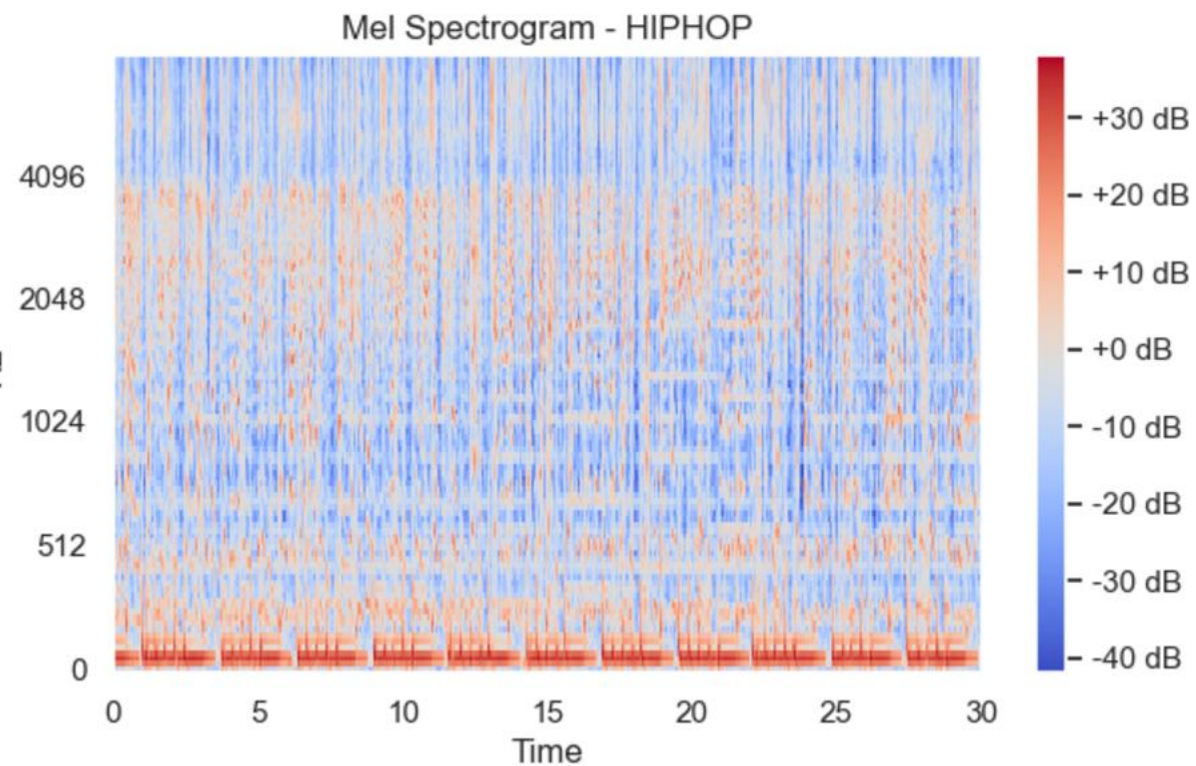
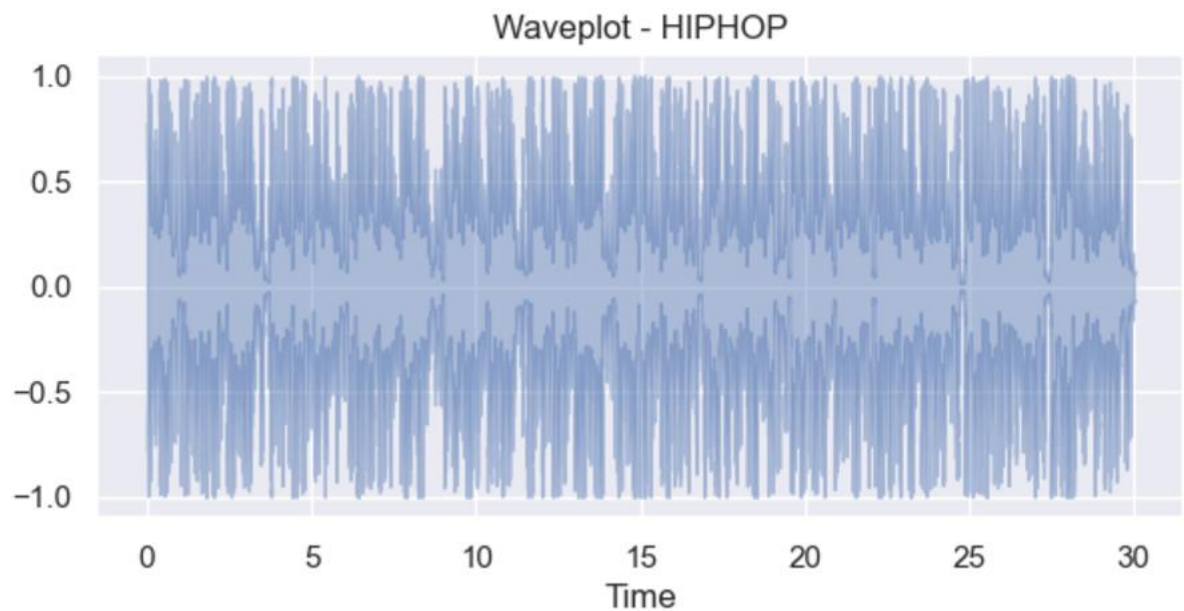
4. Disco



Exploratory Data Analysis

21

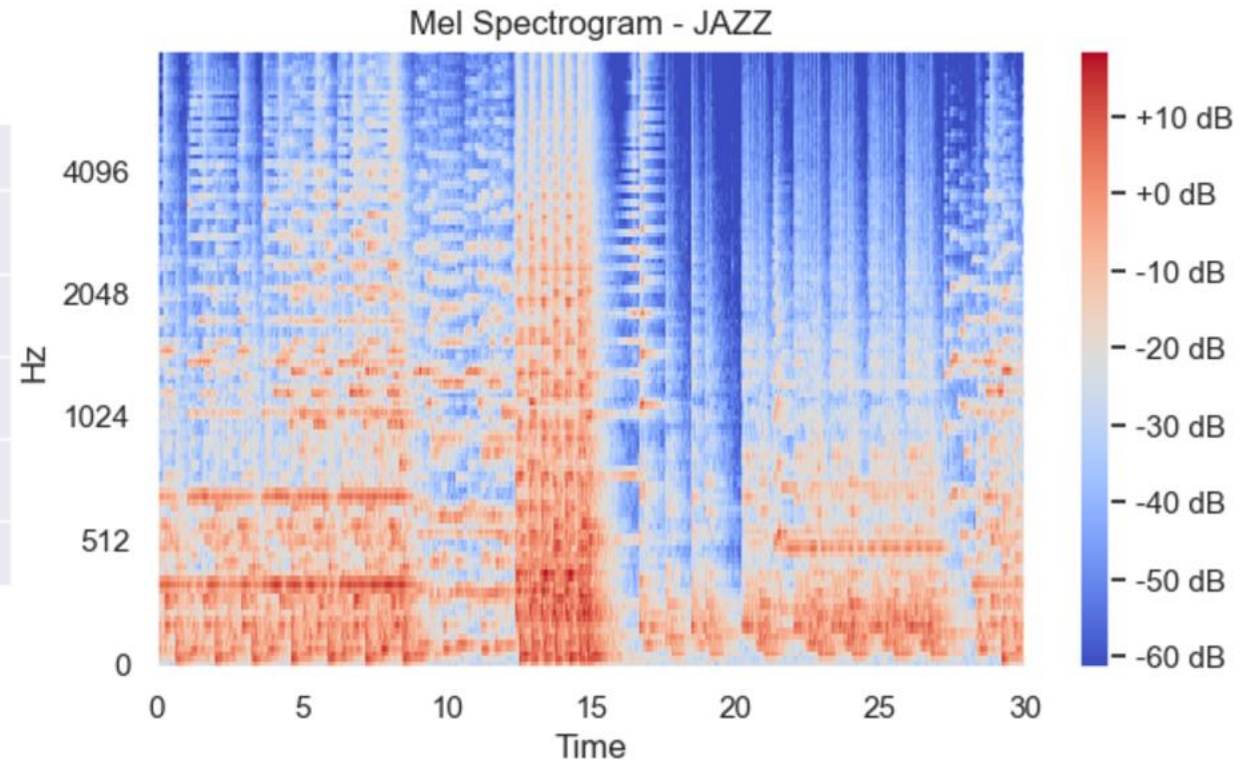
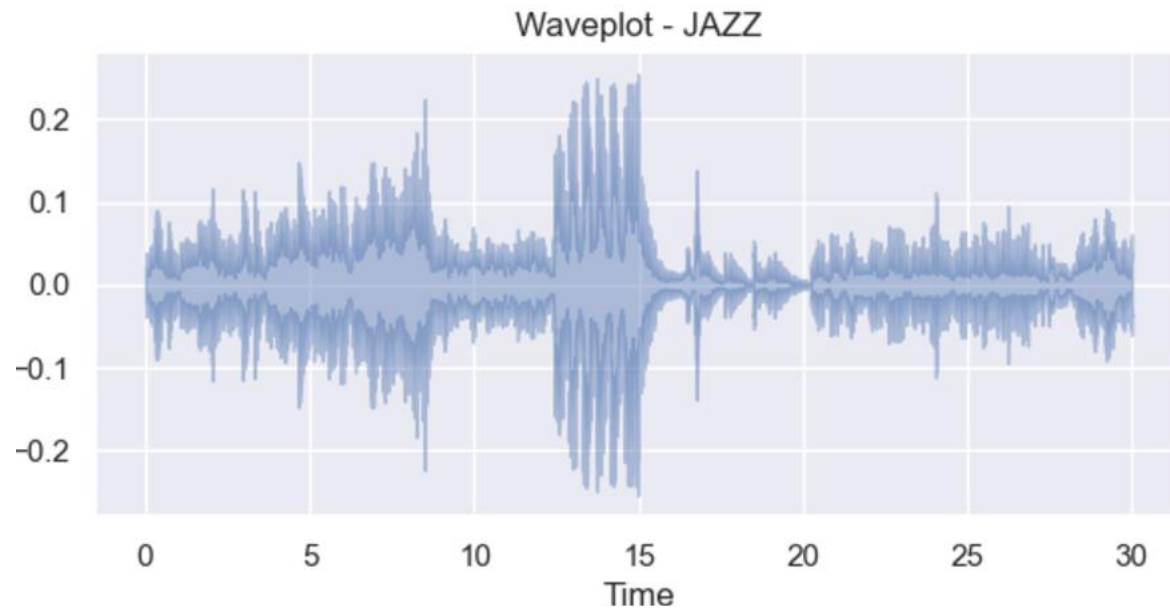
5. Hip-Hop



Exploratory Data Analysis

22

6. Jazz

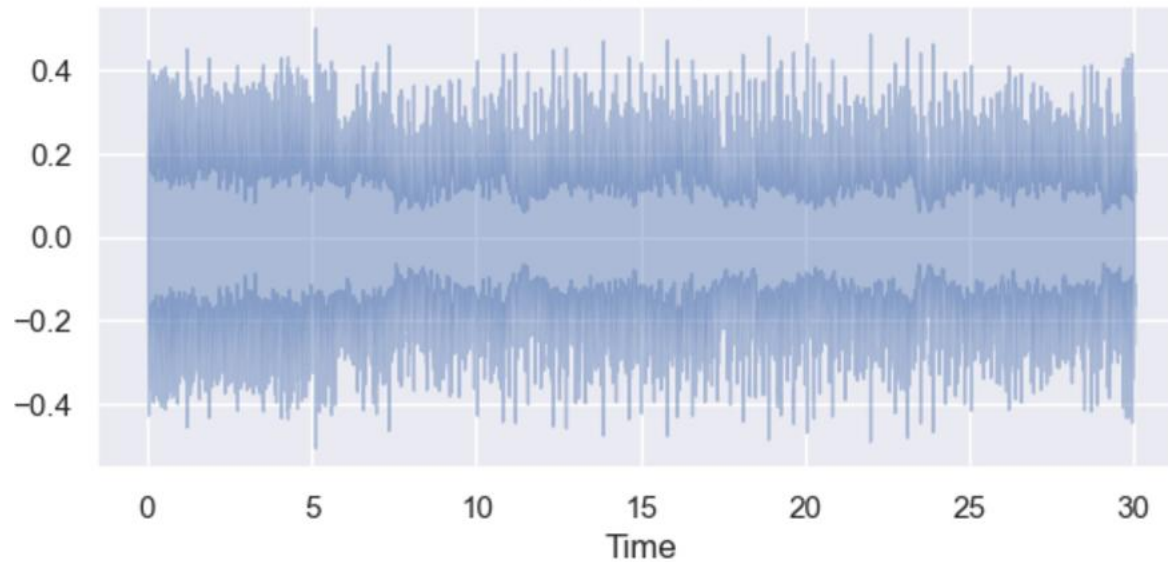


Exploratory Data Analysis

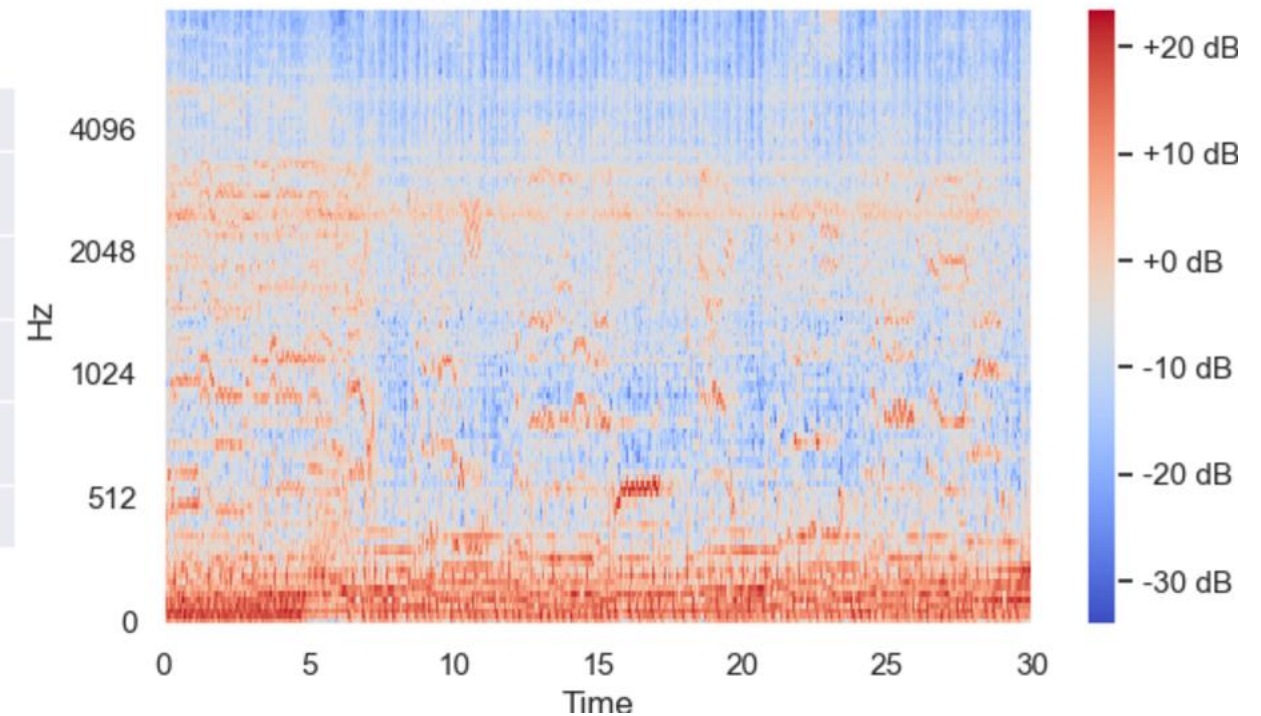
23

7. Metal

Waveplot - METAL



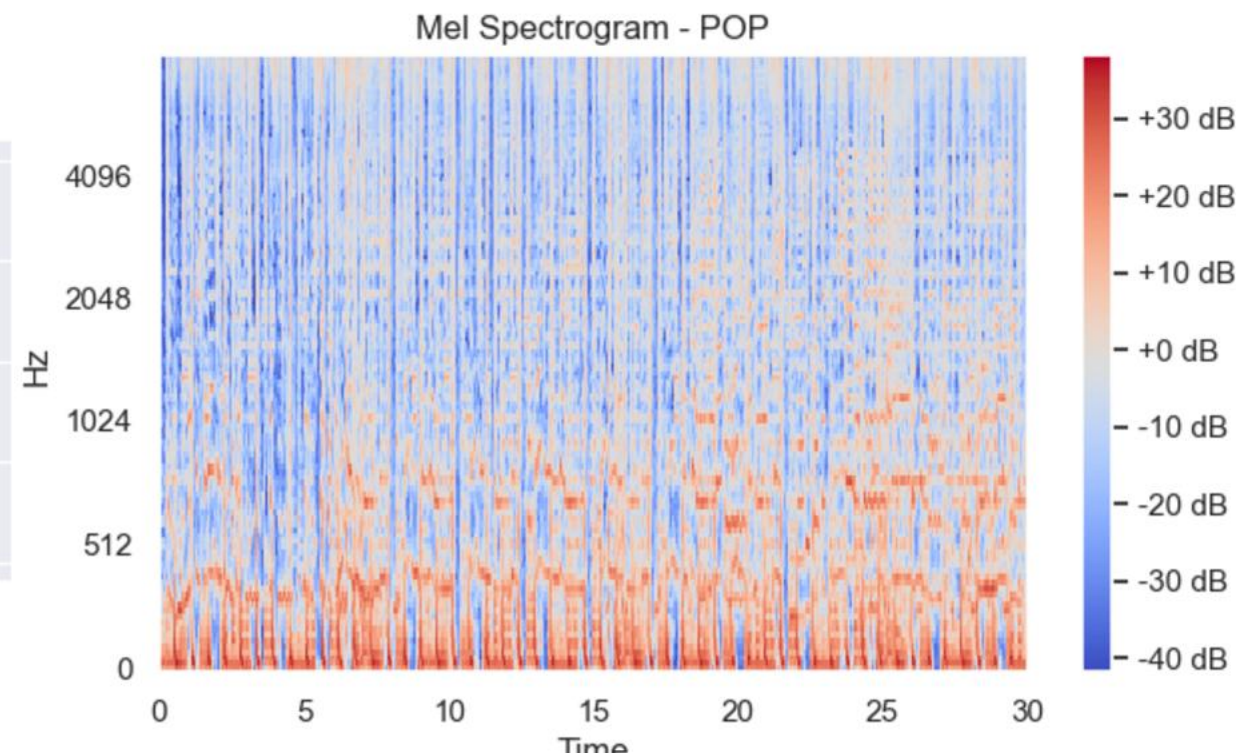
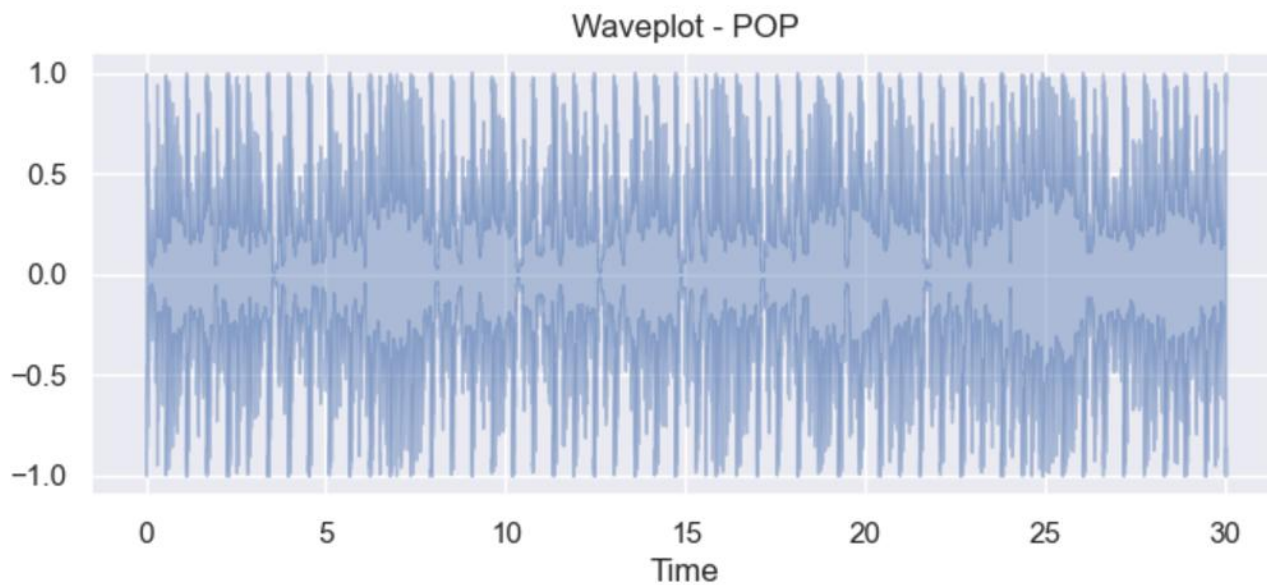
Mel Spectrogram - METAL



Exploratory Data Analysis

24

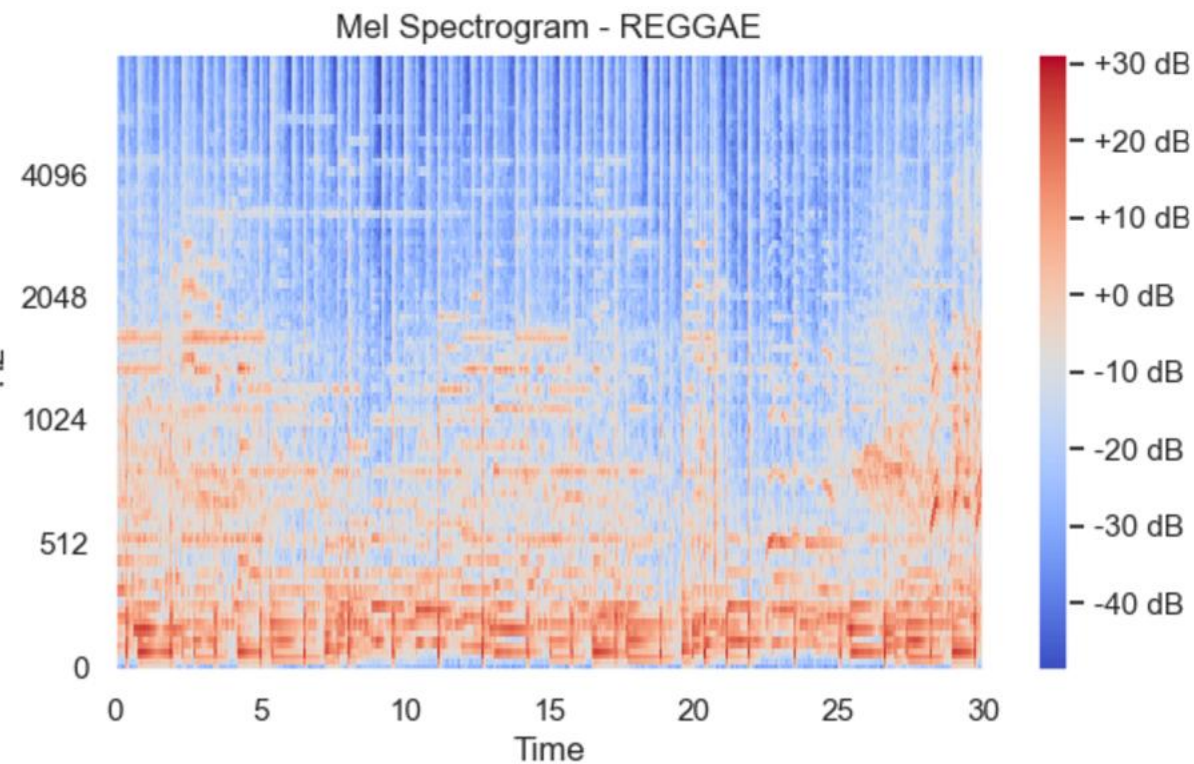
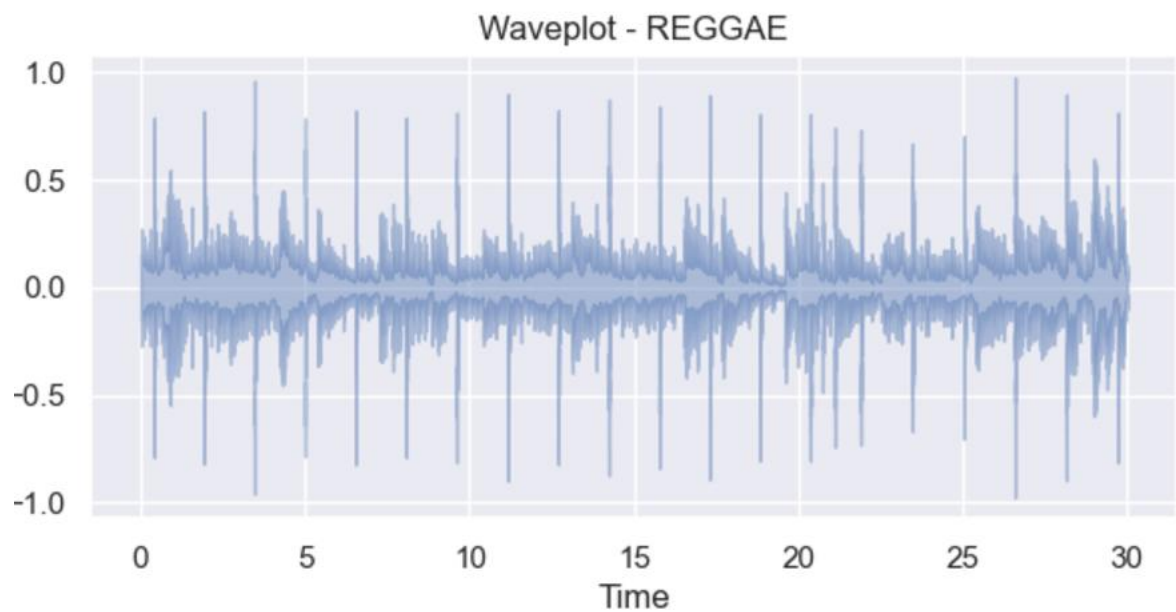
8. Pop



Exploratory Data Analysis

25

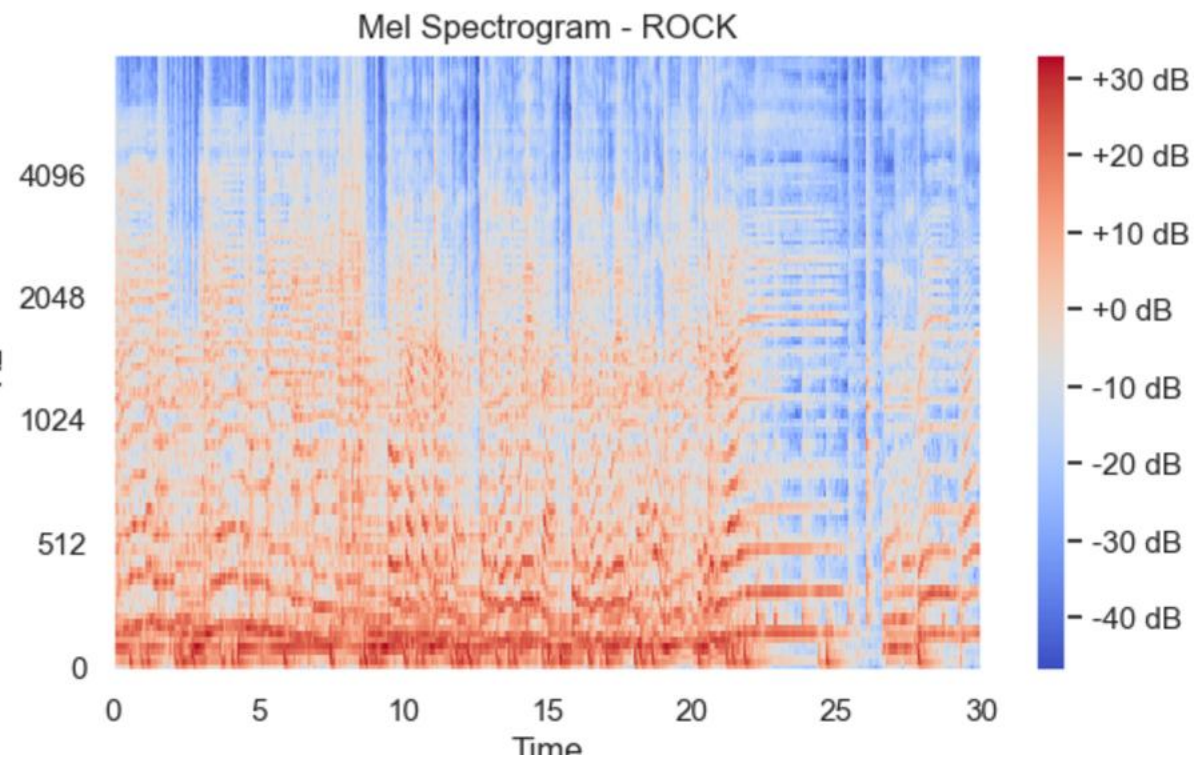
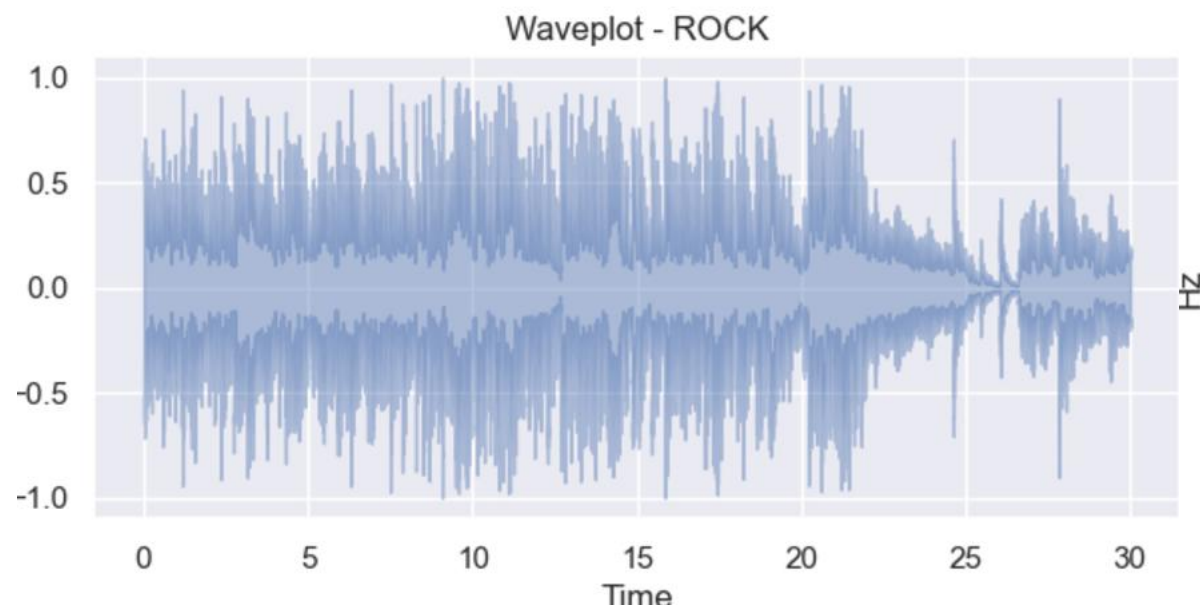
9. Reggae



Exploratory Data Analysis

26

10. Rock



Train and Test Data

27

To create the X and y of this dataset, it is needed to process the data, extract the features and assign the labels.

```
path = 'Data/genres_original'
```

Process the data.

```
def process_data(data_path):
    X = []
    y = []

    for genre in os.listdir(path):
        genre_path = os.path.join(path, genre)
        if os.path.isdir(genre_path):
            print(f"Processing {genre} files...")

            for file_name in os.listdir(genre_path):
                if file_name.endswith('.wav'):
                    file_path = os.path.join(genre_path, file_name)
                    extracted_features = extract_features(file_path)

                    if extracted_features:
                        X.append(extracted_features)
                        y.append(genre)

    return np.array(X), np.array(y)
```

Extract the features.

```
def extract_features(file_path):
    try:
        audio, sr = librosa.load(file_path)

        mfccs = librosa.feature.mfcc(y=audio, sr=sr, n_mfcc=13)
        spectral_centroid = librosa.feature.spectral_centroid(y=audio, sr=sr)
        chroma = librosa.feature.chroma_stft(y=audio, sr=sr)

        features = []
        for feature in [mfccs, spectral_centroid, chroma]:
            features.extend([
                np.mean(feature),
                np.std(feature),
                np.max(feature),
                np.min(feature)
            ])

        return features
    except Exception as e:
        print(f"Error extracting features from {file_path}: {str(e)}")
        return None
```

Train and Test Data

28

```
X, y = process_data(path)
```

```
Processing blues files...  
Processing classical files...  
Processing country files...  
Processing disco files...  
Processing hiphop files...  
Processing jazz files...  
Processing metal files...  
Processing pop files...  
Processing reggae files...  
Processing rock files...
```

```
print(X.shape, y.shape)
```

```
(999, 12) (999,)
```

Split the data.

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2, random_state = 42)
```

```
print(X_train.shape, y_train.shape)
```

```
(799, 12) (799,)
```

```
print(X_test.shape, y_test.shape)
```

```
(200, 12) (200,)
```

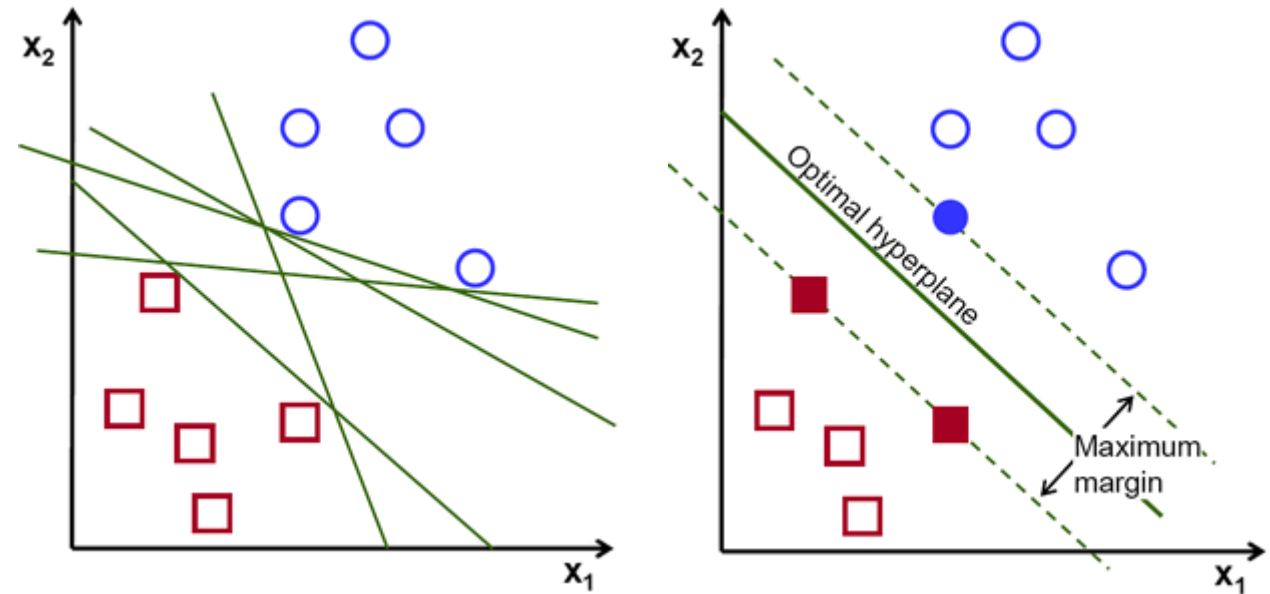


Support Vector Machine

Support Vector Machine (SVM)

30

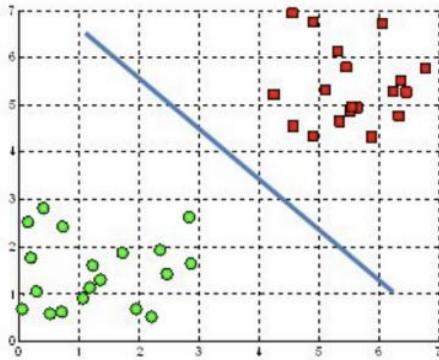
- SVMs are a set of supervised learning methods that can be used for classification, regression and detecting outliers. They find the most similar examples between classes – *support vectors*.
- Each data item is plot in an n -dimensional space (n is the number of features), performing classification by finding the *hyperplane* that differentiates the classes.
- Works well for classifying higher-dimensional data (datasets with lots of features).
- Finds higher-dimensional support vectors which “divide” the data.
- Applies *kernels* to represent data in higher-dimensional spaces to find hyperplanes that might not be apparent in lower dimensions.



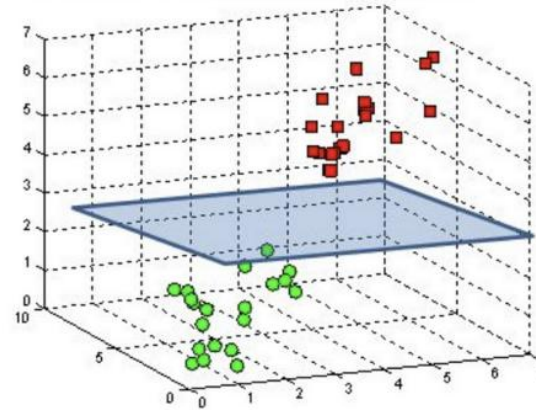
Support Vector Machine (SVM)

31

A hyperplane in $\mathbb{R}^2$ is a line

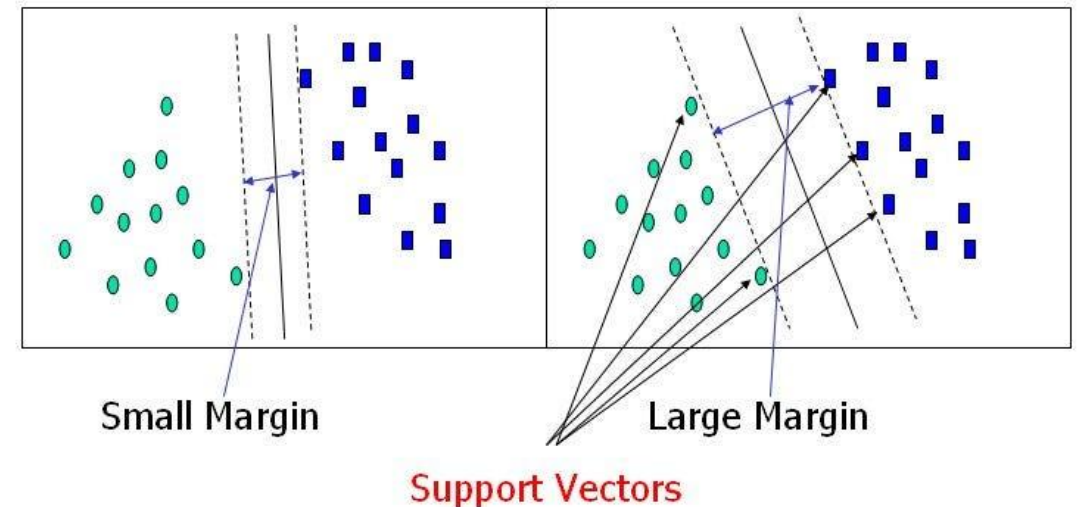


A hyperplane in $\mathbb{R}^3$ is a plane



- The dimension of the hyperplane depends upon the number of features. If the number of input features is 2, then the hyperplane is just a line. If the number of input features is 3, then the hyperplane becomes a two-dimensional plane.

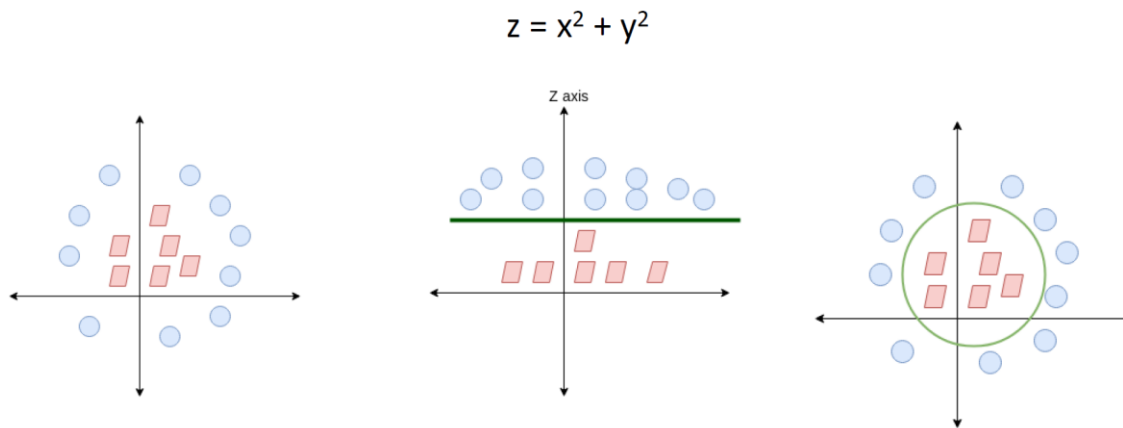
- Hyperplanes are decision boundaries that help classify the data points.
- Data points falling on either side of the hyperplane can be attributed to different classes.



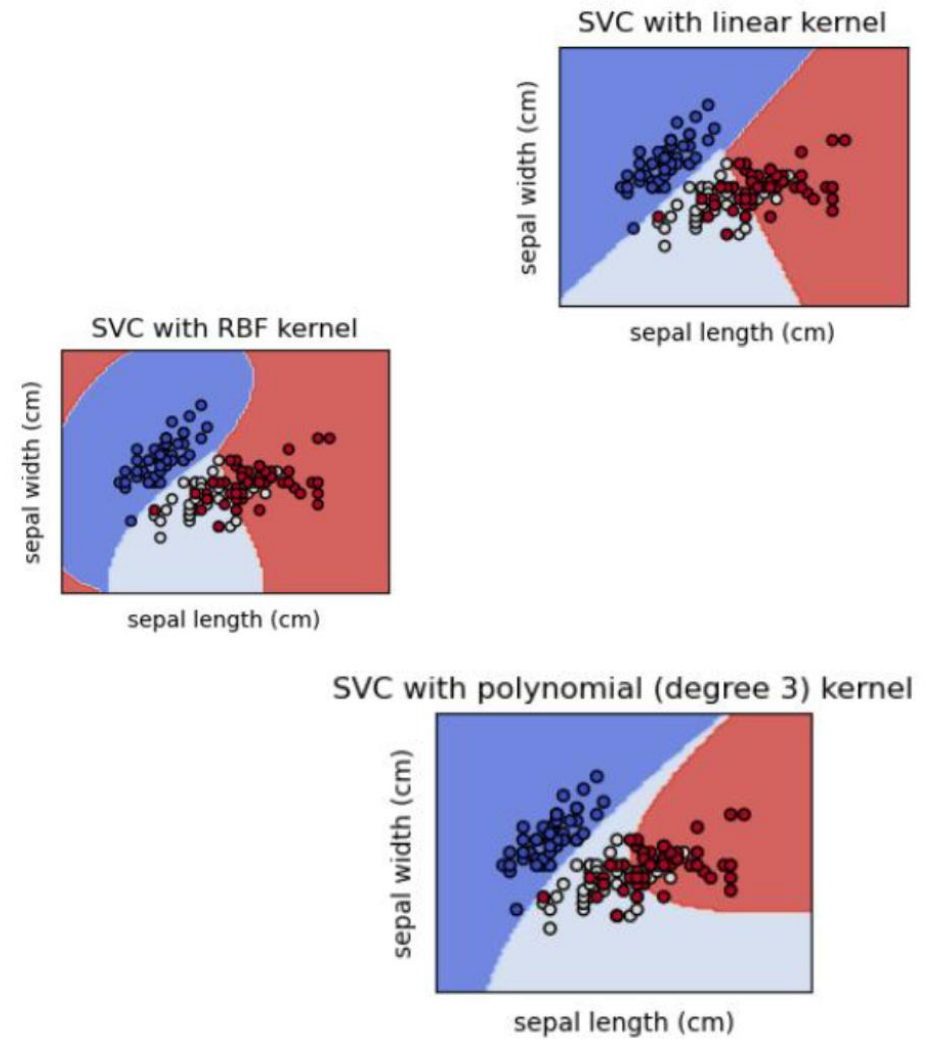
Support Vectors

Kernel

32

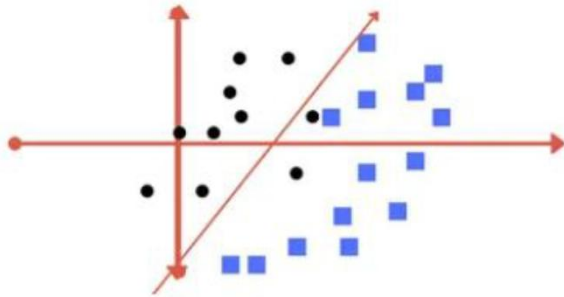


This transformation
is called kernel

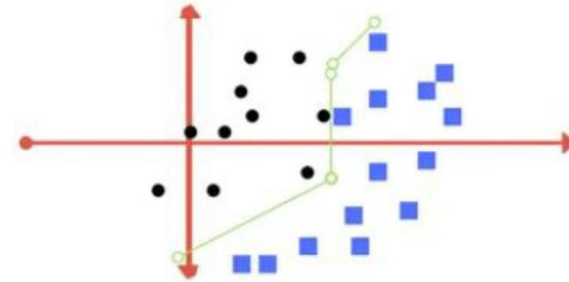


Regularisation - C

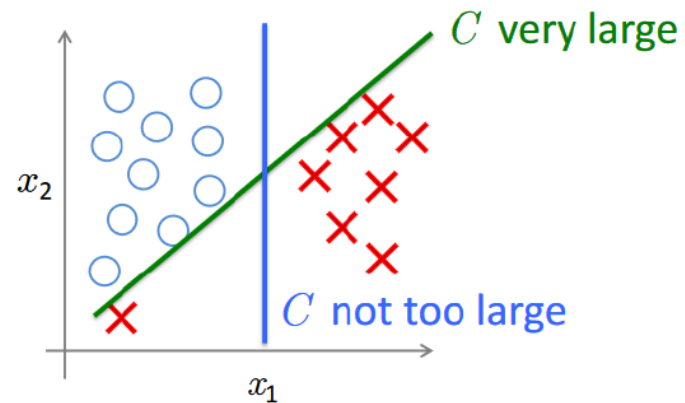
33



low regularisation (C) > larger margin

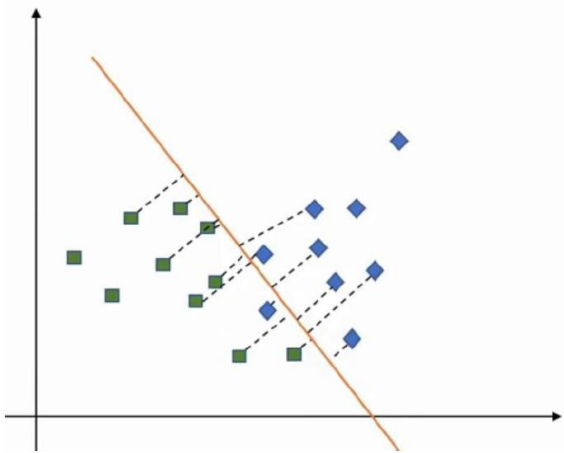


high regularisation (C) > smaller margin

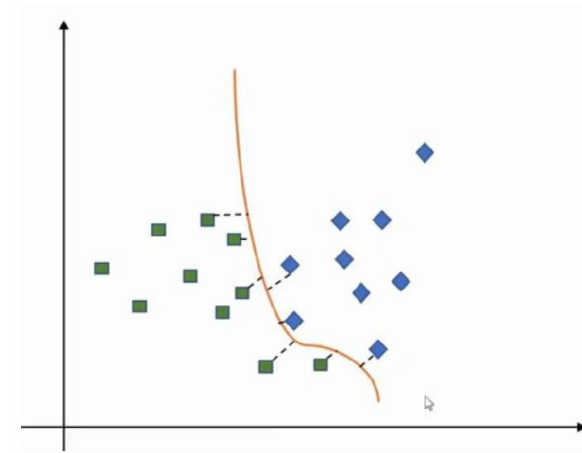


Gamma

34



low gamma > points far from hiperplane



high gamma > points close to hiperplane

Support Vector Machine (SVM)

35

Advantages	Disadvantages
Effective in high-dimensional spaces	If the number of features is much greater than the number of samples, selecting kernel functions and regularisation term can help avoid overfitting Do not provide probability estimates directly; these are calculated using expensive 5-fold cross-validation.
Effective in cases where the number of dimensions $>$ the number of samples	
It uses a subset of training points in the decision function (support vectors), making it memory efficient	
Versatile because different kernel functions can be specified for the decision function. Common kernels are provided, but custom kernels can also be specified.	

Support Vector Classifier

36

`class sklearn.svm.SVC(*, C=1.0, kernel='rbf', degree=3, gamma='scale', coef0=0.0, shrinking=True, probability=False, tol=0.001, cache_size=200, class_weight=None, verbose=False, max_iter=-1, decision_function_shape='ovr', break_ties=False, random_state=None)`

```
classifier = SVC(kernel='rbf', random_state = 42)
```

```
classifier.fit(X_train, y_train)
```

▼ SVC ⓘ ?

SVC(random_state=42)

Obtain the scores.

```
print("Training set score: {:.3f}".format(classifier.score(X_train, y_train)))  
print("Test set score: {:.3f}".format(classifier.score(X_test, y_test)))
```

Training set score: 0.390

Test set score: 0.320

Support Vector Classifier

37

Obtain the predictions.

```
y_pred = classifier.predict(X_test)
```

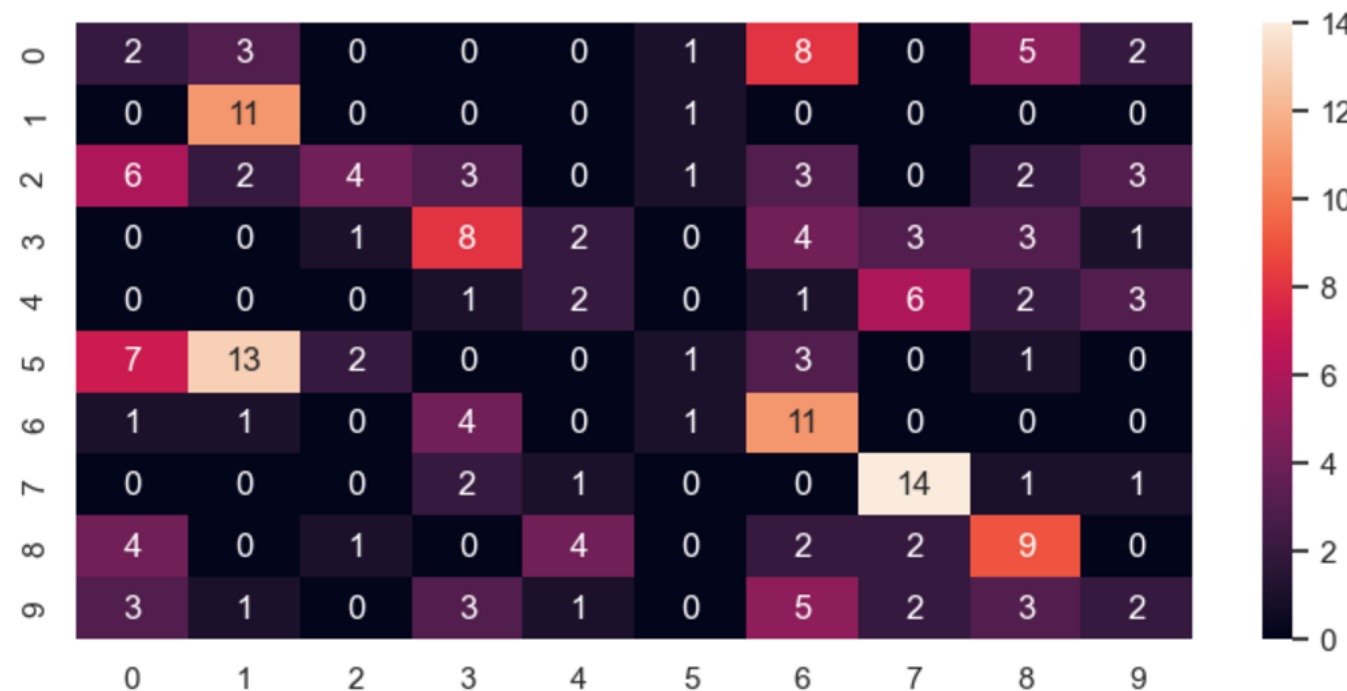
Evaluate the model.

```
cm = confusion_matrix(y_test, y_pred)
```

```
sns.set(rc = {'figure.figsize':(9, 4)})  
sns.heatmap(cm, annot=True)  
print(classification_report(y_test, y_pred))
```

```
['blues': 0, 'classical': 1, 'country': 2,  
 'disco': 3, 'hiphop': 4, 'jazz': 5 ,  
 'metal': 6, 'pop': 7, 'reggae': 8,  
 'rock': 9]
```

	precision	recall	f1-score	support
blues	0.09	0.10	0.09	21
classical	0.35	0.92	0.51	12
country	0.50	0.17	0.25	24
disco	0.38	0.36	0.37	22
hiphop	0.20	0.13	0.16	15
jazz	0.20	0.04	0.06	27
metal	0.30	0.61	0.40	18
pop	0.52	0.74	0.61	19
reggae	0.35	0.41	0.38	22
rock	0.17	0.10	0.12	20
accuracy			0.32	200
macro avg	0.31	0.36	0.30	200
weighted avg	0.31	0.32	0.28	200



Support Vector Classifier

38

Select an audio file.

```
path_test = 'Data/genres_original/hiphop/hiphop.00004.wav'
```

```
audio_test, sr = librosa.load(path_test)
```

```
ipd.Audio(audio_test, rate=sr)
```

▶ 0:00 / 0:30 — 🔊 ⋮

How can the model be improved?

Apply the SVC to that file.

```
a_test=[]
mfccs = librosa.feature.mfcc(y=audio_test, sr=sr, n_mfcc=13)
spectral_centroid = librosa.feature.spectral_centroid(y=audio_test, sr=sr)
chroma = librosa.feature.chroma_stft(y=audio_test, sr=sr)
features = []
for feature in [mfccs, spectral_centroid, chroma]:
    features.extend([
        np.mean(feature),
        np.std(feature),
        np.max(feature),
        np.min(feature)
    ])
a_test.append(features)
```

```
print(classifier.predict(np.array(a_test)))
```

```
['reggae']
```



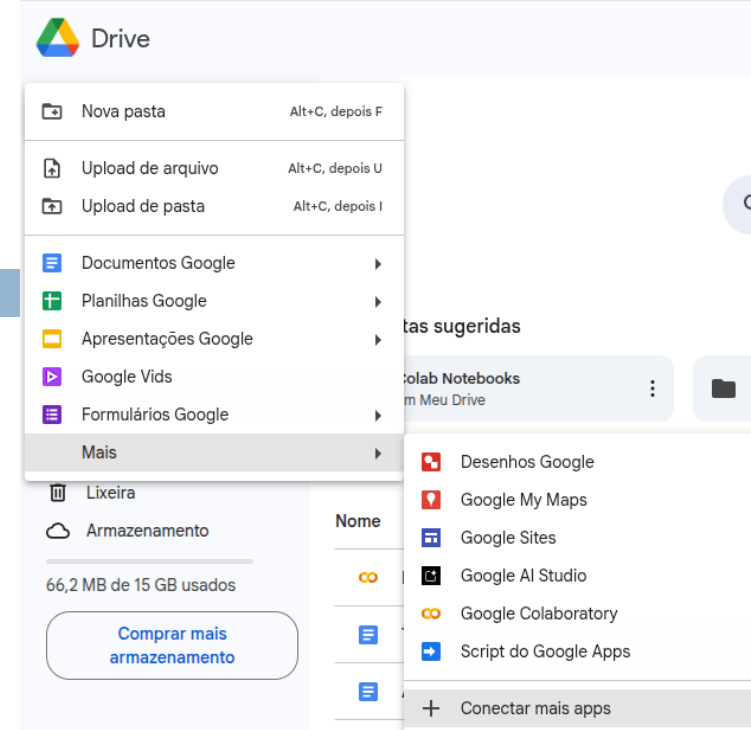
Hands On

AI Assistant as a Helper

40

Setup guide

1. Install the Colab add-on (one-time setup)
 - Open Google Drive (drive.google.com).
 - Click on **+ New** (top left corner) → **More** → **Connect more apps**.
 - In the search bar, type "Colaboratory" (look for the orange/yellow "CO" logo).
 - Click **Install** to add it to your drive.
2. Open the shared notebook
 - Click the link below to access the workbook: <https://shorturl.at/FLs4f>
3. Save your personal copy (the shared file is read only)
 - Go to the top menu: **File** → **Save a copy in Drive**.
 - A new tab will open with your personal copy.
 - Close the old tab and work in the new one.
4. Using the AI Assistant (LLM)
 - Colab has a built-in LLM (Gemini) to help you code.
 - Click the "Gemini" or Sparkle icon at the bottom of the page.
 - You can use the chat to:
 - Generate code (e.g., "Write a function to plot data...").
 - Explain code (e.g., "Explain print() function").
 - Fix errors (e.g., Click "Explain Error" when a cell fails).



Produtos pagos do Colab - Cancelar contratos

